Modèles Algorithmiques Fondamentaux sur Structures à Accès Direct en C++

Thème principal : Parcours complets et recherche séquentielle sur tableaux et chaînes de caractères

Synthèse exécutive : Ce cours établit les schémas algorithmiques de base indispensables en BUT 1 Informatique pour manipuler des structures de données à accès direct (tableaux croisières/statiques, std::vector, std::string). L'enjeu est de maîtriser la distinction fondamentale entre les itérations complètes (parcours systématique et parcours conditionnel) et les itérations partielles (recherche de première occurrence avec arrêt précoce). La maîtrise de ces patron de conception algorithmiques garantit l'écriture d'un code C++ optimisé, lisible et exempt de débordements mémoire (out-of-bounds).

1. Le Contexte & La Notion d'Accès Direct

En programmation imperative C++, une structure à accès direct comme un tableau fixe ou un std::string permet d'atteindre n'importe quelle cellule immédiatement en connaissant son adresse relative (l'indice). Cependant, traiter une collection d'éléments nécessite une méthode rigoureuse. On distingue deux grands besoins : soit nous devons examiner CHAQUE élément du tableau (par exemple pour calculer une moyenne ou afficher le contenu), soit nous devons chercher une information précise et S'ARRÊTER dès qu'elle est trouvée. Utiliser une mauvaise boucle (par exemple une boucle FOR complète pour chercher un élément situé en première position) entraîne un gaspillage de ressources processeur et constitue une faute de conception algorithmique pénalisée en examen.

- Structure à accès direct : Structure de données où chaque élément est accessible en temps constant O(1) via son indice (ex: tab[i]).
- Indice et Bornes : En C++, les indices varient strictement de 0 à N-1 pour une taille N.
- Enjeu algorithmique : Adapter la structure de contrôle (FOR vs WHILE) selon le besoin de parcourir la totalité ou une partie des données.

Cas concret enseignant : *L'enseignant a illustré l'accès direct avec l'image d'un casier postal : si vous avez 100 casiers numérotés de 0 à 99, vous pouvez ouvrir directement le casier n°42 sans ouvrir les 41 précédents. Si vous cherchez une lettre urgente et que vous la trouvez dans le casier n°2, vous ne continuez pas à ouvrir les 97 casiers suivants !*

2. Modèle 1 : Parcours Complet avec Traitement Systématique

Ce modèle est utilisé lorsque chaque élément de la structure contribue au résultat final. C'est le cas typique du calcul de la somme des éléments d'un tableau d'entiers, de l'affichage de tous les caractères d'une chaîne, ou de la recherche du maximum/minimum global. La syntaxe standard repose sur une boucle pour (for) avec un compteur `i` initialisé à 0, une condition d'arrêt stricte `i < taille`, et un incrément `i++` à chaque tour.

- Objectif : Appliquer la même opération à 100% des éléments de la structure.
- Structure de contrôle : Boucle `for` déterminée de 0 à N-1.
- Invariable : Le nombre d'itérations est connu d'avance (exactement N).

Cas concret enseignant : *Le professeur a pris l'exemple du calcul de la moyenne de la promo : pour calculer la moyenne globale de R1.01, l'enseignant est obligé de lire la note de CHAQUE étudiant sans en sauter un seul. Le traitement est appliqué systématiquement.*

3. Modèle 2 : Parcours Complet avec Traitement Conditionné

Le parcours reste complet car il faut visiter chaque cellule du tableau, mais le traitement interne n'est pas exécuté systématiquement. On intègre un test logique `if (condition)` à l'intérieur de la boucle. Si la condition est vraie, l'action est réalisée (ex: incrementer un compteur de notes supérieures à 10) ; si elle est fausse, on passe

directement à l'élément suivant. La boucle continue impérativement jusqu'au dernier élément.

- Objectif : Examiner tous les éléments mais n'agir que sur ceux qui satisfont un prédicat (condition logique).
- Structure de contrôle : Boucle `for` enveloppant une structure conditionnelle `if`.
- Cas d'usage : Dénombrement (comptage), filtrage, cumul sélectif.

Cas concret enseignant : *Exemple de l'appel en amphi : le prof parcourt la liste de tous les étudiants (parcours complet), mais n'ajoute un point de bonus que si l'étudiant a rendu son TP à l'heure (traitement conditionné).*

4. Modèle 3 : Recherche de Première Occurrence (Parcours Partiel)

Contrairement aux parcours complets, la recherche de première occurrence doit s'arrêter dès que l'élément cible est identifié. L'utilisation d'une boucle `for` est ici proscrite au profit d'une boucle `while`. La condition de continuation doit combiner deux aspects : 1) On n'a pas encore dépassé la taille du tableau (`i < taille`), ET 2) On n'a pas encore trouvé l'élément (`!trouve` ou `tab[i] != cible`). Attention : en C++, il faut TOUJOURS tester la limite de l'indice `i < taille` EN PREMIER dans le `&&` pour éviter d'accéder à `tab[i]` lorsque `i` atteint `taille` (segmentation fault ou lecture mémoire interdite).

- Objectif : Localiser le premier élément vérifiant une propriété et s'interrompre immédiatement.
- Structure de contrôle : Boucle `while` avec condition composée (non trouvé ET non fin de tableau).
- Drapeau (Boolean flag) ou double condition : `(i < taille && !trouve)`.
- Piège mortel : L'évaluation courte-circuit en C++ (Short-circuit evaluation).

Cas concret enseignant : *Metaphore du prof : Chercher ses clés dans son sac. Dès qu'on touche les clés, on arrête de fouiller. Continuer à fouiller alors qu'on a déjà les clés en main est absurde.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi est-il interdit d'utiliser un 'break' dans une boucle 'for' pour s'arrêter quand on a trouvé l'élément ?
Réponse du professeur : Même si le compilateur C++ l'accepte, l'utilisation de 'break' casse la structuration algorithmique propre. Une boucle 'for' annonce à la lecture du code un nombre d'itérations fixe et connu. Si l'itération est conditionnée par une recherche, la norme académique exige une boucle 'while' où la condition d'arrêt apparaît clairement dans l'en-tête.

Question étudiant : Que se passe-t-il si j'écris `while (tab[i] != cible && i < taille)` au lieu de `while (i < taille && tab[i] != cible)` ?
Réponse du professeur : C'est une erreur classique qui coûte cher en partiel ! En C++, l'opérateur && est évalué de gauche à droite avec évaluation courte-circuit. Si l'élément n'est pas dans le tableau, 'i' va valoir 'taille'. Dans votre version incorrecte, C++ va d'abord évaluer `tab[taille]` (accès hors mémoire autorisée), provoquant un crash du programme avant même d'avoir pu vérifier `i < taille`.

■ Exercice & Démonstration d'Application

Énoncé : Écrire une fonction C++ nommée `trouverPremierNegatif` qui prend en paramètre un tableau d'entiers `tab` et sa taille `taille`, et qui retourne l'indice du premier nombre strictement négatif trouvé. Si aucun nombre négatif n'est présent, la fonction doit retourner -1.

Résolution pas à pas :
Étape 1 : Initialisation de l'indice `i = 0` et d'un booléen `trouve = false`. Étape 2 : Écriture de la boucle `while` contrôlant les bornes `i < taille` et l'état `!trouve`. Étape 3 : Dans le corps de la boucle, si `tab[i] < 0`, on passe `trouve` à `true`. Sinon, on incrémente `i`. Étape 4 : Après la boucle, si `trouve` est vrai, on retourne `i`, sinon on retourne -1.

Résultat : #include int trouverPremierNegatif(const int tab[], int taille) { int i = 0; bool trouve = false; while (i < taille && !trouve) { if (tab[i] < 0) { trouve = true; } else { i++; } } if (trouve) { return i; // Indice du premier

négatif } else { return -1; // Aucun négatif trouvé } } // Exemple d'exécution : avec {5, 8, -3, 4, -1}, renvoie 2.

■ Auto-Évaluation Flash d'Examen

Q : Quelle structure de boucle doit-on privilégier pour un parcours complet systématique ?

R : La boucle 'for' (pour), car le nombre d'itérations est fixe et connu à l'avance (de 0 à N-1).

Q : Dans une recherche de première occurrence, pourquoi la condition `i < taille` doit-elle être placée avant le test sur l'élément du tableau ?

R : Grâce à l'évaluation courte-circuit du '&&' en C++ : si `i < taille` est faux (fin de tableau), le deuxième test n'est pas évalué, ce qui évite un accès hors-bornes mémoire (`out of bounds`).

Q : Quelle est la valeur retournée par convention lors d'une recherche d'indice si l'élément n'existe pas dans le tableau ?

R : La valeur -1 (ou la taille N du tableau selon les conventions), car -1 n'est jamais un indice valide en C++.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765